

CSIS 4495 — Applied Research Project

Project Proposal

MedGuard: A Zero-Trust IoMT Health Data Pipeline for Real-Time Anomaly Detection and Automated Device Isolation

Team Members

Jude Enuanwa — Student ID: 300403859 **(Team Lead)**

Oghenefejiro Agwere — Student ID: 300395874

Course: CSIS 4495 — Section: 002

Douglas College — Post-Baccalaureate Diploma in Emerging Technology (CIS)

Date: September 15, 2026 (revised September 21, 2026)

1. Introduction

1.1 Domain Overview and Background

The Internet of Medical Things (IoMT) refers to the growing network of connected medical devices — infusion pumps, cardiac monitors, imaging equipment, and wearable patient sensors — that collect and transmit physiological data for continuous clinical monitoring. IoMT adoption has accelerated rapidly across hospitals and remote-care settings because it enables earlier disease detection, reduces manual charting, and supports personalized, data-driven treatment. However, this connectivity introduces a security surface that traditional IT infrastructure was never designed to defend.

Most IoMT devices operate with limited processing power, memory, and battery capacity, which makes conventional security controls such as heavyweight encryption or endpoint agents difficult or impossible to deploy directly on the device. Combined with the sensitivity of the data involved and the direct link between device integrity and patient safety, this makes IoMT an unusually high-stakes environment: a compromised infusion pump or a spoofed cardiac monitor reading is not merely a data breach, it is a potential patient-safety incident.

1.2 Problem Framing

This project addresses the following research question: can a real-time monitoring pipeline reliably detect anomalous or malicious activity in IoMT network traffic and API payloads using unsupervised machine learning, while enforcing zero-trust access principles at the gateway layer, without requiring changes to the medical devices themselves?

This framing matters for two reasons. First, IoMT devices are frequently legacy hardware that cannot be patched or reconfigured, so any viable security solution has to sit at the network/gateway layer rather than on the device. Second, because new and zero-day attack patterns are common in this space, signature-based detection is insufficient. The system needs to recognize deviations from normal behavior even when the specific attack pattern has never been seen before, which is precisely the strength of unsupervised anomaly detection.

1.3 Literature Review

Si-Ahmed, Al-Garadi, and Boustia (2022) survey machine-learning-based intrusion detection systems for IoMT and note that conventional cryptographic security measures are difficult to implement on medical equipment due to heterogeneous communication protocols and constrained computation, storage, and energy budgets, and that these traditional protections are largely ineffective against novel or zero-day attacks. Their survey frames ML-based intrusion detection as a complementary layer of defense suited to IoMT's specific constraints, which directly motivates this project's choice of an ML-driven detection engine rather than a purely rule-based one.

Reji, Pranggono, Marchang, and Shenfield (2023) propose and evaluate a machine-learning-based anomaly detection approach for IoMT against a realistic public dataset, reflecting the broader research trend of validating IoMT anomaly detectors against benchmark network-traffic datasets rather than purely synthetic data — an approach this project follows by combining CICIDS2017/EU IoMT traffic data with Synthea-generated clinical payloads.

More recent work on ensemble anomaly detection using the CICIoMT2024 dataset highlights that IoMT devices are especially vulnerable to DDoS, denial-of-service, spoofing, and reconnaissance attacks because of their

minimal on-device processing power, and argues that machine learning is well suited to handling the high dimensionality and heterogeneity of IoMT traffic data. Separately, research on zero-trust IoT environments observes that zero-trust enforcement depends on continuous, risk-based access decisions rather than perimeter trust, and that machine-learning-based anomaly detection can serve directly as the risk-scoring component within a zero-trust pipeline — precisely the architectural role this project assigns to its ML engine.

Across this literature, a consistent gap emerges: most published work evaluates detection accuracy in isolation, with comparatively little attention paid to how a detection result should be operationalized in real time, i.e. how a risk score actually triggers an alert or a device-isolation action inside a working pipeline. This project's contribution is not a novel detection algorithm, but an applied, end-to-end system that connects zero-trust ingestion, unsupervised detection, and an actionable real-time alerting/isolation interface into a single working pipeline closing that gap at a project scope appropriate for a two-person, one-semester build.

1.4 Initial Hypotheses and Expected Benefit

- Unsupervised models (Isolation Forest, Autoencoders) trained on normal IoMT traffic patterns will detect injected anomalous/attack traffic with meaningfully higher recall than a static, rule-based threshold approach.
- A zero-trust gateway (token auth + schema validation) enforced before data reaches the detection engine will reduce the volume of malformed or unauthorized payloads that reach the ML layer, improving signal quality for the anomaly detector.
- A 0–100 risk score is more actionable for a SOC/IT administrator than a binary flag, because it supports tiered responses (e.g., log only, alert, or auto-isolate) rather than a single blunt action.
- The primary benefit of this research is a demonstrable, portfolio-level proof of concept showing how zero-trust principles and unsupervised ML can be combined into a practical medical-device security tool, applicable beyond the classroom to real healthcare IT security discussions.

2. Proposed Research Project

2.1 Research Design, Objectives, and Methodology

This project follows a design-science / applied-engineering methodology rather than a purely experimental one: the primary deliverable is a working system (MedGuard), evaluated against standard intrusion-detection metrics. The objectives are to (1) build a zero-trust ingestion gateway for simulated IoMT telemetry, (2) train and evaluate unsupervised anomaly detection models against known-attack traffic, (3) expose real-time risk scores and alerts through a dashboard, and (4) implement a one-click device isolation action as the system's response mechanism.

This methodology is justified by the applied nature of the course and by the literature reviewed above, which consistently evaluates IoMT anomaly detection in the context of a deployable system rather than as an abstract classification problem. Building an end-to-end pipeline not just a trained model is what allows the project to be evaluated on both detection accuracy and real-world usability (latency, alert clarity, and operator response time).

2.2 Data Collection and Analysis

- Synthetic clinical data: generated locally using Synthea to produce FHIR/HL7-shaped patient records and simulated IoMT device telemetry (e.g., infusion pump rates, cardiac monitor readings).

- Attack/traffic data: CICIDS2017 and/or the ECU IoMT dataset, both public, labeled network-traffic datasets containing normal traffic plus labeled attack classes (DoS, spoofing, reconnaissance, etc.), used to train and evaluate the anomaly detector.
- Sample size: the full CICIDS2017 dataset contains several million labeled flow records; a representative, class-balanced subset will be sampled for training within the compute constraints of the team's local machines.
- Analysis techniques: feature extraction from raw traffic/payload fields, unsupervised model training (Isolation Forest, then Autoencoders), and evaluation using confusion matrices, precision, recall, F1-score, and ROC curves against the labeled attack classes withheld from training.

2.3 Technology Stack

Category	Technology
Operating System / Platform	Docker (Linux containers) on Windows via WSL2; deployable to any Docker-compatible host
Programming Language(s)	Python (data pipeline, ML models, gateway); JavaScript/TypeScript (frontend)
Framework(s)	FastAPI (gateway/backend), Kafka via the kafka-python-ng client (stream processing), Scikit-Learn / PyTorch (ML), React Native / Flutter / Dash (frontend, TBD)
Database	PostgreSQL (device metadata, risk score history, alert acknowledgment/verification logs)
Front-end	MedGuard dashboard — SOC view (risk charts, alert feed, isolation control) and a simplified nurse/clinical-staff view (device tips, acknowledge action)
Back-end	FastAPI zero-trust gateway, Kafka stream pipeline, WebSocket push notifications (SOC and nurse channels)
AI/LLM Integration	Claude or a comparable LLM API, used for a single grounded summarization call over queried Postgres event logs to generate incident reports
Containerization	Docker & Docker Compose (Kafka, Zookeeper, Postgres, gateway service)

2.4 Expected Results

We expect the trained Isolation Forest and Autoencoder models to distinguish normal IoMT traffic from injected attack traffic with strong recall on the labeled evaluation set, producing a 0–100 risk score that meaningfully separates benign from malicious sessions. We expect the zero-trust gateway to successfully reject a majority of malformed or unauthenticated payloads before they reach the detection engine, and we expect the end-to-end pipeline (from ingestion to dashboard alert) to operate with low enough latency to be usable as a near-real-time monitoring tool. Practically, this contributes a reusable, demonstrable pattern for combining zero-trust ingestion with unsupervised anomaly detection that extends beyond this course project into general IoT/OT security contexts.

2.5 Evaluation Scheme

Model validity and accuracy will be evaluated quantitatively using a held-out test split of the CICIDS2017/ECU IoMT dataset: precision, recall, F1-score, and ROC/AUC curves computed against the labeled attack classes, as outlined in the project's Week 8–9 penetration-testing and model-tuning phase. Utility will be evaluated qualitatively by simulating payload-tampering and high-frequency traffic attacks against the running pipeline and confirming that (1) the risk score rises appropriately, (2) a WebSocket alert is dispatched to the dashboard, and (3) the one-click isolation action correctly halts further ingestion from the flagged device.

2.6 Extended Ecosystem Design

Since drafting the initial scope, the team refined MedGuard's user model from a single SOC/auditor audience to two distinct audiences sharing the same underlying pipeline, without adding new infrastructure. The same 0–100 risk score and event log are routed differently depending on what triggered the alert: a confirmed behavioral anomaly (an ML-flagged attack pattern) routes to the SOC dashboard with full technical detail and the one-click isolation control, while a structural device-configuration weakness (e.g., auto-lock disabled, default credentials unchanged) checked directly at the gateway, independent of the ML engine routes to the assigned nurse/clinical staff member as a simplified, plain-language remediation tip.

Both channels require acknowledgment rather than being fire-and-forget, and acknowledgment is logged to the same Postgres-backed event table regardless of audience. For nurse-routed configuration alerts specifically, the system goes a step further than a self-reported fix: because the device's own subsequent telemetry already reports its configuration state on every message, the gateway re-checks that state on each new reading and marks the alert `verified_resolved_at` automatically once the flagged setting shows the secure value; a verified remediation record rather than a trusted self-report.

The second addition is an agentic reporting feature: a backend endpoint queries recent flagged events from Postgres and prompts an LLM (Claude or a comparable API) to summarize them into a written incident report, grounded strictly in the queried data. This is intentionally scoped as a single well-grounded summarization call rather than a multi-step agent framework, which keeps it achievable within the existing timeline while still demonstrating an applied LLM-assisted workflow over the system's own operational logs.

2.7 Regulatory & Compliance Considerations

Beyond the FDA and Health Canada premarket cybersecurity guidance named in the original project brief, two further standards inform the system's design. FDA Section 524B of the FD&C Act (effective March 2023) defines a "cyber device" as a device that includes software, connects to the internet directly or indirectly, and has characteristics that make it vulnerable to cyberthreats; qualifying manufacturers must provide a machine-readable Software Bill of Materials and a postmarket vulnerability-management plan. MedGuard's design mirrors the spirit of that vulnerability-management expectation, even though this student project is not itself a regulatory submission.

IEC 81001-5-1:2021 defines life-cycle security requirements specifically for health software, written to support conformance with IEC 62443-4-1 (the broader industrial/networked-control-system security standard family). The two-tier alerting and human-in-the-loop acknowledgment design in Section 2.6 is a direct, practical instance of the life-cycle security activities these standards call for, giving the final report concrete architecture to point to rather than only a general compliance statement.

On data privacy specifically: PIPEDA (federal) and BC's Personal Information Protection Act (PIPA) govern real personal health information. Because MedGuard uses only synthetic Synthea-style clinical data and simulated device telemetry (no real patient data is ever ingested) the system does not, in its current prototype form, fall under PIPEDA/PIPA obligations. This is a deliberate scoping decision rather than an oversight, and the final report will state explicitly what a production deployment handling real PHI would additionally require under those Acts.

2.8 Progress to Date

As of this proposal's submission, the following components are implemented and verified working end-to-end, ahead of the Week 4 target for token validation:

- Full Docker Compose infrastructure running: Kafka, Zookeeper, PostgreSQL, Kafka UI, and the FastAPI gateway service.
- The zero-trust /ingest endpoint: bearer-token authentication and Pydantic schema validation (a discriminated model per device type, with extra fields strictly rejected) are both implemented and confirmed, via automated tests, to correctly reject unauthorized and structurally tampered requests while passing through structurally valid but behaviorally anomalous readings for the ML layer to evaluate.
- Config-vulnerability detection: a rule set flags insecure device settings (auto-lock disabled, default credentials unchanged, outdated firmware, TLS disabled) at ingestion time, confirmed working against simulated devices.
- An IoMT telemetry simulator that authenticates and posts through the gateway (rather than writing to Kafka directly), so the zero-trust layer is genuinely exercised rather than bypassed, with injected attack traffic (out-of-range vitals, traffic floods, payload tampering) and a locally logged ground-truth file for later model evaluation.
- Verified in the Kafka UI: validated messages, including attached vulnerability flags and remediation tips, are landing correctly in the `iomt.telemetry.raw` topic.
- Not yet started: the stream preprocessing pipeline and ML model training (David's Week 4–5 items), the frontend dashboard, and Synthea itself has not yet been run locally — the simulator above currently substitutes for it and is scheduled to be reconciled with real Synthea output before Week 4 closes.

3. Project Planning and Timeline

3.1 Team Responsibilities

Rather than splitting strictly by layer, remaining work is assigned so both team members get hands-on implementation time across all three core areas — cybersecurity/backend, machine learning, and frontend, with a named Primary owner per feature and a Support owner who pairs on design/review. Jude Enuanwa (Team Lead) leads the gateway's authentication and schema validation, config-vulnerability detection, and the SOC dashboard, while supporting David's ML and nurse-view work; David leads TLS/encryption, stream preprocessing, Isolation Forest training, the LLM-based report agent, and the nurse dashboard view, while supporting Jude's gateway pen-testing and SOC view. Model evaluation, WebSocket wiring, penetration testing, and the regulatory/compliance section are done jointly by both members in the same session. Docker/Docker Compose infrastructure and joint-submission coordination remain Jude's responsibility as Team Lead. A detailed feature-by-feature breakdown, including required skills and research topics per task, is maintained in a separate team task-split document.

3.2 Timeline and Milestones

Week	Dates	Phase / Focus	Key Deliverables
1	Sep 9 – 15	Setup & Schema Definition	GitHub repo, Docker environment, CICIDS2017/ECU IoMT dataset ingested, Synthea generator configured
2	Sep 16 – 22	Traffic Simulation & Gateway	Python IoMT telemetry simulation scripts; draft zero-trust gateway validation rules
3	Sep 23 – 29	Proposal Finalization	In-class check-in (Sep 23); Project Proposal submitted, collaborators added to repo (Sep 28)
4	Oct 3 – 9	Stream Preprocessing & Auth	HL7/FHIR cleaning pipeline (Pandas/SQL); zero-trust token validation & encryption implemented
5	Oct 10 – 16	ML Training & Progress Report 1	Isolation Forest / Autoencoder trained on IoMT traffic; risk-scoring algorithm (0–100); Progress Report 1 due Oct 12
6	Oct 17 – 23	MedGuard UI Development	Frontend dashboard built; FastAPI/WebSocket channels connected to detection engine
7	Oct 24 – 30	Mid-Course Check-in & Demo	Real-time WebSocket alerts integrated; live demo (Oct 28); Mid-Course Report & working implementation due Oct 26
8	Oct 31 – Nov 6	Penetration Testing & Tuning	Simulated payload tampering & high-frequency attacks; precision/recall/F1 evaluation
9	Nov 7 – 20	Regulatory Mapping & Progress Report 2	Confusion matrices, ROC curves; FDA/Health Canada guidance mapping; Progress Report 2 due Nov 16
10	Nov 21 – 27	One-Click Isolation & Progress Report 3	Device isolation trigger finalized in MedGuard UI; Progress Report 3 due Nov 23
11	Nov 28 – Dec 4	System Polish & Final Check-in	End-to-end flows finalized; 15–20 page Applied Research Report drafted; final check-in (Dec 2); Progress Report 4 due Nov 30
12	Dec 5 – 9	Final Deliverables	Final report, documentation, and code base due (Dec 7); Final Presentation & live demo (Dec 9)

3.3 Project Management Approach

The team will track work using a Kanban-style board (GitHub Projects) with columns for Backlog, In Progress, Review, and Done, mapped against the 12-week timeline above, which functions as the project's Gantt-style schedule. Each week's deliverables in Section 3.2 double as milestones on the board; in-class check-ins (Sep 23, Oct 28, Dec 2) serve as the project's formal review gates.

4. Project Contract

We, the undersigned, agree to the scope of work, timelines, and individual responsibilities outlined in Sections 2 and 3 of this proposal. We agree to hold a joint status meeting at least once per week (via Discord/in-person) to review progress, and to update our individual work logs after every work session, as required.

We agree that Jude Enuanwa serves as Team Lead and is responsible for all joint submissions (Proposal, Midterm Report, Final Report). Progress Reports remain individual submissions per team member as required by the course.

Signed: _____ Date: _____

Jude Enuanwa (Team Lead)

Signed: _____ Date: _____

Oghenefejiro Agwere

5. AI Use Section

The following table documents the specific AI tools used in preparing this proposal and the associated project scaffolding, per course requirements. Full prompt history is included in the Appendix.

AI Tool Name	Version, Account Type	Specific Feature / Use	Value Addition
Claude (Anthropic)	Claude Sonnet 4.6/5 (free version)	Scaffolded the Docker Compose infrastructure and starter code; implemented and tested the gateway's /ingest endpoint (auth, Pydantic schema validation, Kafka publish) and config-vulnerability rules; rewrote the device simulator to route through the gateway; diagnosed and fixed a kafka-python/Python 3.12 compatibility bug; drafted the cross-functional task-split document and this proposal; troubleshoot local Docker/WSL2 setup issues.	All architectural decisions, technology choices, ecosystem design (SOC/nurse alert routing, verified-resolution loop, report agent), and the project scope itself were made by the team; Claude was used to accelerate implementation, testing, and documentation. All generated code was reviewed, run, and independently tested (including automated checks confirming auth rejection, schema rejection, and correct pass-through behavior) before being treated as final.

Appendix A — AI Prompt History

The following is a representative summary of the prompts used with Claude (Anthropic) during project setup and proposal drafting. Full chat logs are retained by the team and available on request.

- "Do you understand what's happening here? How do I go about starting this project and implementing features?" — used to plan an initial technical approach and Week 1 setup steps from the project brief.
- "How do I setup docker compose?" — used to scaffold a docker-compose.yml (Kafka, Zookeeper, Postgres, Kafka UI, FastAPI gateway) and a minimal gateway skeleton.
- "I've never used Kafka before, why did you use it?" , "Explain to me the entire relationship, Docker and Kafka." — used for conceptual understanding, not code generation.
- "My terminal doesn't recognize docker" and subsequent troubleshooting prompts — used to resolve local Docker Desktop/WSL2/virtualization setup issues on a Windows machine.
- "I want to have everything setup so David can start work on his part." — used to scaffold stream/ and ml/ starter files, a shared .env.example, and a teammate onboarding document.
- "I want another person such as the nurse using the device to get a notification too..." and follow-up questions on acknowledgment and verification — used to design the two-audience (SOC/nurse) alert routing and the verified-resolution loop described in Section 2.6.
- "Split the remaining tasks between David and I... each one of us must touch each part of a module instead of just having one person work on it" — used to produce a separate cross-functional task-split document, summarized in Section 3.1.
- "Prep what I need for week 2-4, let's implement if we can" — used to implement and test the gateway's /ingest endpoint, config-vulnerability rules, and the updated device simulator, as described in Section 2.8.
- "Why are we checking who sent it?" and "how do we set which devices are allowed to or not... in such a way it can't be falsified" — used for conceptual understanding of authentication vs. schema validation; no code was generated from these prompts.
- "localhost:8000 shows nothing" — used to diagnose and fix a kafka-python/Python 3.12 compatibility bug (switched to kafka-python-ng) that was preventing the gateway container from starting.
- "from the pasted file i just wanna understand the concept of the project" — provides the concept stripped down to essentials.
- "rather than go deeper what i'd wanna understand is the working, what and how would these software programs help perform the task" — specifically talks about each piece of software and how it actually *works* internally (the mechanics).
- "the instructions on how to set things up for my laptop, i need you to break it down so i can do it" — setup broken into a clear sequence you can just follow along with.

6. Work Date/Hours Logs

Per course requirements, entries below reflect work completed to date and will be updated frequently, in small increments, as work is performed — not batched into large end-of-week entries.

Student: Jude Enuanwa

Date	Hours	Description of Work Done
Sep 9, 2026	1.5	Reviewed project brief, set up GitHub repository structure and initial planning.
Sep 12, 2026	2	Researched Docker/Kafka architecture for the ingestion pipeline; drafted docker-compose.yml.
Sep 14, 2026	1.5	Resolved local Docker Desktop/WSL2/virtualization setup issues; verified full stack running.
Sep 15, 2026	2	Scaffolded gateway, stream, and ml starter folders; drafted this Project Proposal.
Sep 20-21, 2026	3	Designed the extended ecosystem (SOC/nurse alert routing, verified-resolution loop, report agent); implemented and tested the gateway's /ingest endpoint, schemas, auth, and config-vulnerability rules; rewrote the device simulator to route through the gateway; diagnosed and fixed a Kafka client compatibility bug; drafted the task-split document and revised this proposal.

Student: Oghenefejiro Agwere [David]

Date	Hours	Description of Work Done
Sep 17, 2026	2	Researched the project's core technology stack and architecture to prepare for implementation: reviewed how Docker/Docker Compose, Kafka, Zookeeper, and Postgres function together in the pipeline; studied FastAPI/uvicorn's role as the zero-trust gateway and the mechanics of mTLS/token-based zero-trust auth.
Sep 21, 2026	1.5	Set up local development environment: installed Docker Desktop, cloned the MedGuard repository, and resolved a docker-compose.yml not found error by confirming working directory. Ran docker compose up -d --build to pull and build the Kafka/Zookeeper/Postgres/Kafka-UI images and the gateway service; verified all five containers (zookeeper, kafka, kafka-ui, gateway, iomt-db) reporting Up via docker compose ps. Confirmed gateway health endpoint, Kafka UI, and Postgres connectivity locally.

7. Closing and References

The team acknowledges Douglas College's CSIS 4495 course materials and instructor guidance in shaping the structure and requirements of this proposal.

References

Si-Ahmed, A., Al-Garadi, M. A., & Boustia, N. (2022). Survey of machine learning based intrusion detection methods for Internet of Medical Things. arXiv preprint arXiv:2202.09657. <https://arxiv.org/pdf/2202.09657>

Reji, A., Pranggono, B., Marchang, J., & Shenfield, A. (2023). Anomaly detection for the internet-of-medical-things. In 2023 IEEE International Conference on Communications Workshops (ICC Workshops) (pp. 1944–1949). IEEE. <https://shura.shu.ac.uk/31706>

Enhanced anomaly detection in IoMT networks using ensemble AI models on the CICIoMT2024 dataset. (2025). arXiv preprint arXiv:2502.11854. <https://arxiv.org/pdf/2502.11854>

Advanced anomaly detection and threat intelligence in zero trust IoT environments using machine learning. arXiv preprint arXiv:2604.23332. <https://arxiv.org/pdf/2604.23332>

Costa, G., Forestiero, A., Macrì, D., & Ortale, R. Trustworthy precision medicine: An interpretable approach to detecting anomalous behavior of IoT devices. IOS Press. <https://ebooks.iospress.nl/pdf/doi/10.3233/SHTI240068>

Hsiao, H.-C., Tsai, C.-P., Li, Z.-X., Lee, C.-H., Chen, J.-S., Lai, Y.-C., Wang, J.-C., Li, S.-C., Gao, J.-C., & Lee, Y.-H. (2023). Load balancing algorithms and their impacts on Apache Kafka. In 2023 IEEE International Conference on Big Data (pp. 1726–1735). IEEE. Consulted for background on Kafka partition/broker load-balancing behavior relevant to this project's stream pipeline design.